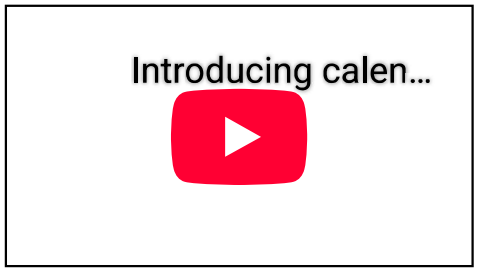



# Introducing calendar-based time intelligence in DAX

(https://www.sqlbi.com/articles/introducing-calendar-based-time-intelligence-in-dax/)

This article introduces the new calendar-based time intelligence functions in DAX, available in preview from the September 2025 release of Power BI.

DOWNLOAD SAMPLE FILES

FEB 4, 2026
UPDATED

Marco Russo & Alberto Ferrari (https://www.sqlbi.com/about/#team)

CALENDAR (HTTPS://WWW.SQLBI.COM/TOPICS/CALENDAR/)
DAX (HTTPS://WWW.SQLBI.COM/TOPICS/DAX/)
TIME INTELLIGENCE (HTTPS://WWW.SQLBI.COM/TOPICS/TIME-INTELLIGENCE/)

Since its first release in 2010, DAX has had a set of time intelligence functions to simplify calculations like year-to-date, year-over-year, and so on. However, the calculations only supported the Gregorian calendar, without addressing similar requirements for other calendars, such as the 4-4-5, ISO, and many other non-Gregorian calendars. With the classic time intelligence, the columns of the *Date* table were unknown to the time intelligence functions, with the only exception of the date column in the *Date* table, typically *Date[Date]*.

The calendar-based time intelligence functions are designed to handle different types of calendars; therefore, they rely on columns in the date table to provide information about when a year begins and ends, as well as how it is divided into quarters, months, and/or weeks. Consequently, the *Date* table must contain several columns with specific meanings, and the developer must associate these columns with levels in the calendar definition. These columns are meaningful to both humans and DAX.

For example, the `SAMEPERIODLASTYEAR` (https://dax.guide/sameperiodlastyear/?aff=sqlbi) function is used to display the value of a measure over the same period in the previous year. In classic time intelligence, the function works by detecting the currently-selected time period, and using the Date column, moving the filter back one year. With calendar-based time intelligence, the *Date* table must contain a specific column indicating the year, which allows the DAX engine to adjust the filter on that column to move it back to the previous year. Things are much more complicated than this, but – at this point – a generic idea of the behavior is more than enough.

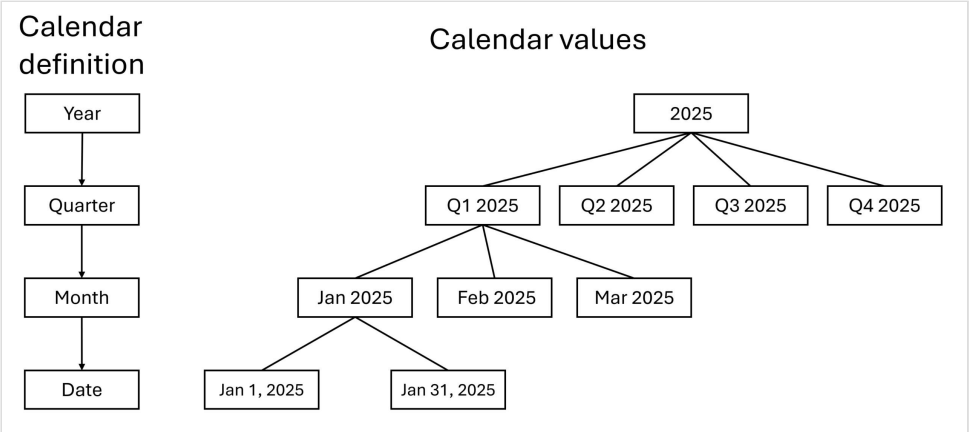
This article does not include the user interface. Please watch the video to see how the user interface works, and refer to the additional links in case we create new videos, should the user interface that defines calendars evolve.

# Introducing calendars

The calendar-based time intelligence does not assume a year is a Gregorian year. The calendar-based functions base their behavior on the content of columns in the *Date* table itself. This is why the *Date* table must include columns to indicate how a year is divided into periods, and developers must label these columns to inform DAX of their meaning. Columns are associated with their meaning by creating one or more *Calendar* objects.

A calendar stores the association between table columns and time levels, such as year, quarter, and month. We can build multiple calendars on the same table. For example, we could include in the same *Date* table a Gregorian calendar, an ISO calendar, and a fiscal calendar. However, including multiple calendars in the same *Date* table poses some challenges because of the way calendar-based time intelligence functions remove filters: all the calendars participate in that activity even when only one calendar is referenced in the function.

Here is a typical hierarchy of a Gregorian calendar, divided into years, quarters, months, and dates. On the left, there are four levels; on the right, a sample of values is shown.




A calendar associates levels of the hierarchy with columns in the *Date* table. This mapping allows developers to create any calendar. For example, creating a calendar with thirteen months is a viable option with calendars, whereas it would not be possible with classic time intelligence.

While 13-month calendars are not frequent, a type of calendar that is very commonly used and requires calendar-based time intelligence functions is the weekly calendar. Indeed, weeks cannot be

aggregated to months or quarters in a regular Gregorian calendar, because there are weeks belonging to two months. Therefore, it is not possible to create a hierarchy that includes weeks and months according to the definition of the Gregorian calendar. However, it is possible to use weeks as the primary calendar entity and to redefine the concepts of years, quarters, and months to establish a natural hierarchy based on weeks, where every week belongs to a single "period" (sometimes called a month) and to a single quarter. We will create a weekly calendar later in this chapter.

## Using calendars

With classic time intelligence functions, the presence of a *Date* column in the *Date* table is enough to make them work. With the calendar-based time intelligence functions, once the *Date* table is in the model, it is still necessary to associate columns with the proper category in calendars.

A single date table can host multiple calendars, and the same column can be used in multiple calendars. Adequate care must be taken to ensure that the column is always associated with the same category. For example, if the *Date[Month]* column is associated with the *Month in Year* category, then the same column cannot be associated with a different category in another calendar.

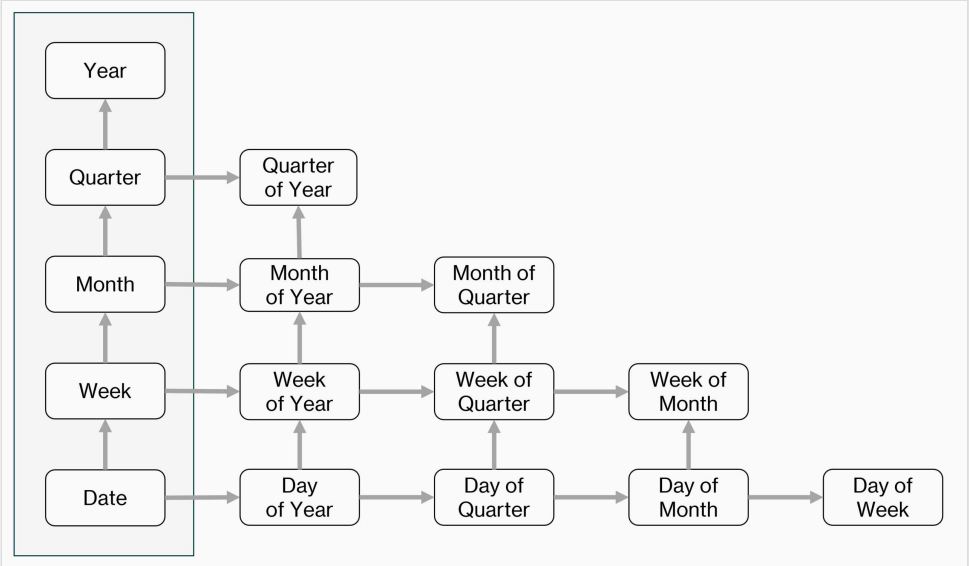
A column category defines the semantics of the column for the calendar and describes the expected behavior of the column related to other columns in the same calendar. The available column categories are listed in the following table.

| **Category** | **Example** | **Description** | **Type** | **Example of cardinality** |
|---|---|---|---|---|
| Year | 2025 | The year | Complete | 1 per year |
| Quarter | Q1 2025 | The quarter, including the year | Complete | 4 per year |
| Month | January 2025 | The month, including the year | Complete | 12 per year |

| Category | Example | Description | Type | Example of cardinality |
|---|---|---|---|---|
| Week | 2025 Week 20 | The week, including the year | Complete | 52/3 per year |
| Date | 01/05/2025 | The individual date | Complete | 365/6 per year |
| Quarter of Year | Q1 | The quarter, without the year | Partial | 4 |
| Month of Year | January | The month, without the year | Partial | 12 |
| Month of Quarter | M2 | The month of the quarter | Partial | 3 |
| Week of Year | Week 51 | The week of the year | Partial | 53 |
| Week of Quarter | Week 11 | The week of the quarter | Partial | 13 |
| Week of Month | Week 3 | The week of the month | Partial | 5 |
| Day of Year | Day 312 | The day, without the year | Partial | 366 |
| Day of Quarter | Day 54 | The day of the quarter | Partial | 92 |
| Day of Month | Day 12 | The day of the month | Partial | 31 |
| Day of Week | Monday | The day of the week | Partial | 7 |

It is not necessary to define all categories. However, it is a good idea to assign a category to the columns in the Date table that participate in the hierarchical navigation in the calendar. Time intelligence functions can work with “incomplete” calendars, where not all the categories have corresponding columns.

There are relationships among the various categories. The following figure illustrates the different categories, along with the relationships. The categories are organized into levels, where each level is part of the upper level. For example, *Quarter* is part of *Year,* and *Month* is part of *Quarter.* Among the many levels, the *Week* is somewhat special because a single week may or may not overlap the month, depending on how the calendar is defined.




The categories highlighted in the first column are the most natural hierarchy: year, quarter, month, week, and date. We refer to this group as the *complete categories*, because each value in the complete categories uniquely identifies a point in time. For example, Q1 2025 clearly identifies a value for *Quarter.* Thus, the column associated with the *Quarter* category must have 20 unique values if the *Date* table contains five years. The categories that do not belong to the group of complete categories are called *partial categories*, because they need two or more categories to identify a period uniquely.

For example, the correct category for a column that has only four unique values, regardless of the year, is *Quarter of Year*, because *Year* is also needed to identify a specific quarter. Indeed, the value Q1 does not define a point in time. It is necessary to specify the value of *Year* to transform Q1 into Q1 2025 and finally obtain a point

in time. The calendar definition is flexible: you can use *Year* and *Quarter of Year* without *Quarter,* or you can use just *Quarter,* or all these categories. The calendar-based time intelligence functions will work in every configuration. They will attempt to minimize the required filter by using a complete category whenever possible, and relying on partial categories when necessary.

Each column category in the calendar has one primary column and a set of associated columns. The primary column is the column that will be included in the result of the calendar-based time intelligence functions. However, there may be multiple columns for the same category. For example, the *Month in Year* category typically contains month names, such as January to December. Still, it is also common to have a column with short month names, like Jan to Dec. Therefore, one column is the primary column, and the other one(s) will be defined as associated columns, which are columns with the same semantics.

Columns tagged in the calendar must be sortable, either directly or through a sort-by-column. Sorting is essential because time intelligence oftentimes requires finding periods before or after the current one. The columns specified in the sort-by-column property are automatically considered as associated columns, even if they are not explicitly mentioned as such.

## Defining calendars

The user interface for defining calendars may change over time, and it can vary depending on the tools used. However, an agnostic way to illustrate how a calendar definition appears is to examine its TMDL representation. For example, the following is the definition of a calendar named Gregorian that includes only complete categories *(Year, Quarter, Month,* and *Date):*

**TMDL definition for a calendar**

```
createOrReplace

    ref table Date

        calendar Gregorian

            calendarColumnGroup = year
                primaryColumn: Year

            calendarColumnGroup = quarter
                primaryColumn: 'Year Quarter Number'
                associatedColumn: 'Year Quarter'

            calendarColumnGroup = month
                primaryColumn: 'Year Month Number'
                associatedColumn: 'Year Month'
                associatedColumn: 'Year Month Short'

            calendarColumnGroup = date
                primaryColumn: Date
```

COPY CONVENTIONS #1 DAX FORMATTER

Every category is a *calendarColumnGroup* object associated with a category and must have one primaryColumn. Every *associatedColumn* has one line for each associated column. All the columns that are included in the sort-by-column property for columns tagged as *primaryColumn* or *associatedColumn* are automatically considered as *associatedColumn* as well. All the other columns of the *Date* table that are not part of any *calendarColumnGroup* object are considered filter-keep columns.

## Introducing time-related columns

When developers use the calendar-based time intelligence functions, the automatic `REMOVEFILTERS` (https://dax.guide/removefilters/?aff=sqlbi) on Date no longer occurs. We will describe the behavior in detail in the next section. However, right now, it is important to acknowledge that filter clearing occurs in a significantly different manner. Calendar-based time intelligence functions operate only on the columns tagged in a calendar, adding and removing filters depending on their semantics.

Any column in *Date* that is not tagged in any calendar is a filter-keep column, whereas all the columns that are tagged are filter-clear columns. Therefore, handling filter-keep columns with the calendar-based time intelligence is much easier compared to the classic time intelligence.

However, there may be columns for which we want to remove the filter forcibly, and these columns may very well not belong to any of the column categories of a calendar. Examples of these columns include the season (e.g., Spring, Summer), the moon phase, and columns containing a textual description of the date, such as "Today", "Tomorrow", "Current Month", and so on.

It is possible to tag those columns in the calendar by adding them to the calendar without associating them with any category level. A column tagged in the calendar, with no category, is a time-related column. Time-related columns undergo some specific handling. Most calendar-based time intelligence functions clear filters from time-related columns, with the notable exception of lateral-shifting functions (`DATEADD` (https://dax.guide/dateadd/?aff=sqlbi) and `SAMEPERIODLASTYEAR` (https://dax.guide/sameperiodlastyear/?aff=sqlbi)), which instead keep filters on time-related columns.

We will describe in detail how time-related columns are handled in the following sections. For now, the only topic that matters is to understand that a time-related column is a column added to a calendar but not associated with any column category. Time-related columns are defined in TMDL with a *calendarColumnGroup* that is not associated with any calendar category and only has one or more columns associated with it. The following example shows a calendar where *Period* and *Season* are two time-related columns for the Gregorian calendar:

**TMDL definition for a calendar**

```
createOrReplace

    ref table Date

        calendar Gregorian

            calendarColumnGroup = year
                primaryColumn: Year

            calendarColumnGroup = monthOfYear
                primaryColumn: 'Month Number'
                associatedColumn: Month

            calendarColumnGroup
                column: Period
                column: Season
```

COPY CONVENTIONS #2 DAX FORMATTER

# Understanding the behavior of time intelligence functions

Time intelligence functions modify the filter context over *Date* to obtain a calculation for a different period. In order to reach their goal, time intelligence functions perform these three steps:

1. They determine the currently-selected time period.
2. They determine at which level the current period needs to be shifted/extended.
3. They extend or shift the current selection as indicated by the parameters.

The parameters of each function may affect the second and the third step.

## Understanding the behavior of calendar-based time intelligence

The calendar-based time intelligence functions do not rely on hardcoded knowledge about the calendar. The structure of a year is defined in the column categories of the calendar. Therefore, it is

more relevant to understand the algorithm used by the functions to determine the new filter and how it applies the new filter to the existing filter context.

We do not want to scare our readers. In most scenarios, the natural behavior of calendar-based time intelligence functions is clear, and these functions produce intuitive results. However, some scenarios may produce unexpected results: we want our reader to be aware of them. Moreover, it is possible to modify the algorithm's behavior by using specific parameters. Calendar-based time intelligence functions are more flexible than classic time intelligence functions. With flexibility, it comes naturally that there is some complexity to understand.

Let us create a measure that computes a measure in the same period of the last year with the calendar-based functions:

**Measure in Sales table**

```
Sales SPLY Calendar =
CALCULATE (
    [Sales Amount],
    SAMEPERIODLASTYEAR ( 'Gregorian' )
)
```

COPY CONVENTIONS #3 DAX FORMATTER

The *Same SPLY Calendar* measure produces the same result as the classic *Sales SPLY* version, as shown in the following figure.

| Year | Sales Amount | Sales SPLY | Sales SPLY Calendar |
|---|---|---|---|
| ⊟ **2019** | **3,550,194.76** | **4,984,304.80** | **4,984,304.80** |
| ⊟ **Q1** | **1,071,467.00** | **1,694,367.17** | **1,694,367.17** |
| January | 449,241.51 | 636,983.88 | 636,983.88 |
| February | 488,375.02 | 788,062.88 | 788,062.88 |
| March | 133,850.46 | 269,320.40 | 269,320.40 |
| ⊞ **Q2** | **566,557.47** | **951,456.11** | **951,456.11** |
| ⊞ **Q3** | **821,002.21** | **1,100,858.91** | **1,100,858.91** |
| ⊞ **Q4** | **1,091,168.09** | **1,237,622.62** | **1,237,622.62** |
| ⊟ **2020** | **769,835.80** | **3,550,194.76** | **3,550,194.76** |
| ⊟ **Q1** | **769,835.80** | **1,071,467.00** | **1,071,467.00** |
| January | 376,480.90 | 449,241.51 | 449,241.51 |
| February | 379,349.67 | 488,375.02 | 488,375.02 |
| March | 14,005.22 | 133,850.46 | 133,850.46 |
| ⊞ **Q2** | | **566,557.47** | **566,557.47** |
| ⊞ **Q3** | | **821,002.21** | **821,002.21** |
| ⊞ **Q4** | | **1,091,168.09** | **1,091,168.09** |
| **Total** | **4,320,030.56** | **8,534,499.57** | **8,534,499.57** |

However, this time the engine has been able to shift February 2020 (29 days) to February 2019 (28 days) without relying on any predefined knowledge about the calendar structure. We need to understand the algorithm better.

Let us focus on the cell of February 2020. The filter context contains three columns: Year=2020, Quarter=Q1, Month=February. Remember that the *Quarter* column is tagged as *Quarter of Year* category, and *Month* is tagged as *Month of Year* category. By using these filters, DAX determines that they define a single point in the calendar lattice: February 2020, which is obtained by combining the *Year* and *Month of Year* categories and corresponds to the same level of the *Month* category as seen previously in the illustration of the calendar levels. This is how the calendar-based time intelligence functions identify the “current” selection in the calendar.

Because the function requires shifting the period one year back, further processing is necessary to determine the corresponding month one year ago. Indeed, it may look trivial at this point to conclude that February 2020 shifted back one year to February 2019. However, as humans, we are biased by the fact that we are deeply familiar with the structure of a calendar. The calendar-based time intelligence functions do not assume the calendar structure. February is just a name, as is 2020. At this point, it is helpful to recall that columns used in calendars must be sortable. The year before 2020 can be easily computed: it is 2019. However, regarding the

month, the algorithm requires an additional step. The algorithm must determine that February is the second month in 2020, and maintain the relative position of February in 2019. To obtain the same period in the previous year, it finds the second month in 2019 is February 2019. Therefore, the new filter context contains February 2019.

This algorithm is known as "same distance from parent". Because the function we used shifts the period at the year level, Year is the parent. The returned month is the month with the same distance from the beginning of the previous year. There are significant consequences to this algorithm; we will examine further details later.

## Understanding filter clearing

So far, `SAMEPERIODLASTYEAR` (https://dax.guide/sameperiodlastyear/?aff=sqlbi) has been able to identify the target filter: February 2019. However, applying the new filter is not enough, because it would conflict with the existing filter context. Therefore, `CALCULATE` (https://dax.guide/calculate/?aff=sqlbi) must remove some filters from the *Date* table. If this were a classic function, it would use a ruthless solution: use `REMOVEFILTERS` (https://dax.guide/removefilters/?aff=sqlbi) over the entire *Date* table. However, we have already discussed that the automatic `REMOVEFILTERS` (https://dax.guide/removefilters/?aff=sqlbi) poses serious issues for the handling of filter-keep columns.

Therefore, calendar-based time intelligence functions use a rather sophisticated algorithm to determine which filters will be removed and which ones will be maintained.

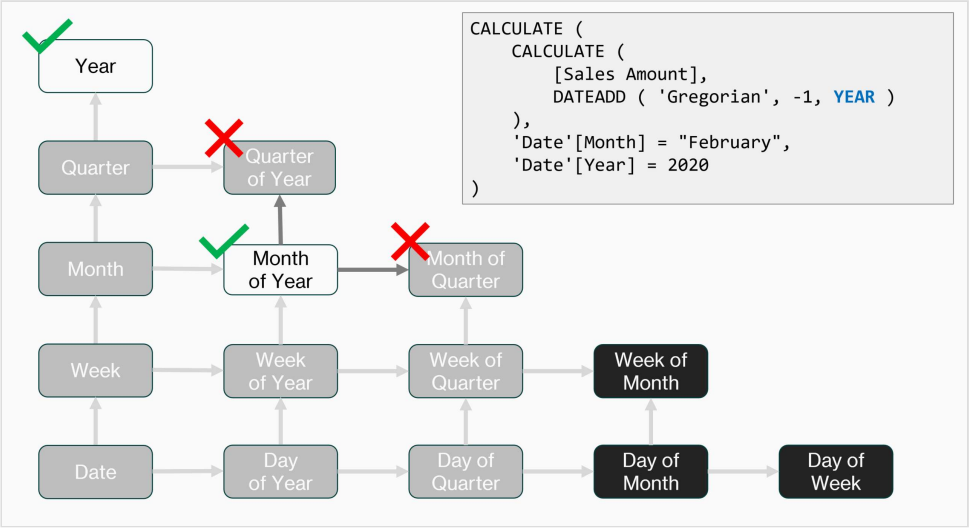
In our example, the new filter context generated by `SAMEPERIODLASTYEAR` (https://dax.guide/sameperiodlastyear/?aff=sqlbi) contains two columns: *Date[Month] = February* and *Date[Year] = 2019*, which are columns tagged as *Month of Year* and *Year* category, respectively. When the calendar-based time intelligence functions evaluate the filter context for each "X" column tagged with a category in the calendar, here is what happens:

- DAX determines the set of column category dependencies of "*X*". To obtain this set, it is enough to follow the arrows in the

diagram. *Year* has no dependencies, while *Month of Year* has both the *Quarter of Year* and *Month of Quarter* as dependencies.

- Once the dependencies are identified, REMOVEFILTERS (https://dax.guide/removefilters/?aff=sqlbi) is applied to all columns above that are in the same vertical position as either *X* or a dependency.

The following illustration shows a graphical representation of this behavior, which shows the underlying DATEADD (https://dax.guide/dateadd/?aff=sqlbi) function invoked by SAMEPERIODLASTYEAR (https://dax.guide/sameperiodlastyear/?aff=sqlbi) in the equivalent DAX code (more on this equivalency later in the chapter). The filters on *Week of Month*, *Day of Month*, and *Day of Week* are not altered, even if present, as those categories represent time-related columns. They are not cleared in this scenario, since they are filter-keep columns.




Filter clearing works across calendars. If the table includes multiple calendars, the filter is removed from any column tagged in any calendar at the levels where the filter clearing happens. In other words, if the *Date* table includes an ISO calendar with an *ISO Month* column tagged as *Month of Year*, the *ISO Month* will receive REMOVEFILTERS (https://dax.guide/removefilters/?aff=sqlbi) in this example, even though it does not belong to the current calendar. This is the reason why creating multiple calendars on the same *Date* table can be confusing. It is possible to create multiple calendars, but developers need to pay special attention to which filters will be removed and which ones will be maintained.

## Understanding lateral shift and hierarchical shift

So far in this chapter, we analyzed two functions: SAMEPERIODLASTYEAR (https://dax.guide/sameperiodlastyear/?aff=sqlbi) and DATESYTD (https://dax.guide/datesytd/?aff=sqlbi). We chose those two functions for a good reason: they represent two different categories of time intelligence functions.

- SAMEPERIODLASTYEAR (https://dax.guide/sameperiodlastyear/?aff=sqlbi) performs a lateral shift, meaning that it considers the current selection in the filter context and moves it back one year. If the original selection is the month level, SAMEPERIODLASTYEAR (https://dax.guide/sameperiodlastyear/?aff=sqlbi) performs filters at the month level, just moving the filter one year back. A lateral shift does not change the granularity of the filter. The only other function that performs lateral shifts is DATEADD (https://dax.guide/dateadd/?aff=sqlbi), which is the function internally used by SAMEPERIODLASTYEAR (https://dax.guide/sameperiodlastyear/?aff=sqlbi).
- DATESYTD (https://dax.guide/datesytd/?aff=sqlbi) performs a hierarchical shift. If the original filter contains only one date, the new filter may need to filter dates, years, and months. DATESYTD (https://dax.guide/datesytd/?aff=sqlbi), as well as any time intelligence function other than DATEADD (https://dax.guide/dateadd/?aff=sqlbi) and SAMEPERIODLASTYEAR (https://dax.guide/sameperiodlastyear/?aff=sqlbi), can change the granularity of the selection and extend or reduce it. For example, DATESYTD (https://dax.guide/datesytd/?aff=sqlbi) usually extends the period selected, whereas NEXTMONTH (https://dax.guide/nextmonth/?aff=sqlbi) always returns one month, even if the initial selection was a quarter or a year.

There is a crucial difference between lateral shift and hierarchical shift in how these functions handle time-related columns. A time-related column is a column tagged in the calendar but not associated with any category in the hierarchy.

When a time intelligence function performs a lateral shift, it keeps the filter on the time-related columns. On the other hand, hierarchical shifts clear the filters on time-related columns. A time-related column has mixed behavior: it is a filter-keep column for lateral shifts, and it is a filter-clear column for hierarchical shifts. We will describe the time-related column behavior further in the new section about `DATEADD (https://dax.guide/dateadd/?aff=sqlbi)` and `SAMEPERIODLASTYEAR (https://dax.guide/sameperiodlastyear/?aff=sqlbi)`.

## Conclusions

In this article and in the related video, you learned the basics of calendar-based time intelligence calculations in DAX. We have seen how to define a calendar in a *Date* table, how to assign the right categories to the columns, and how to use the time intelligence functions with the proper calendar. This is just the beginning; over time, we will provide more examples and tools to improve your productivity in creating calendars and implementing calendar-based time intelligence calculations in your reports.

## Downloads

Insert your email address and press *Download* for access to the files used in this article.

Your email addres

**DOWNLOAD (HTTPS://WWW.SQLBI.COM/WP-CONTENT/UPLOADS/C0276-INTRODUCING-CALENDAR-BASED-TIME-INTELLIGENCE-IN-DAX.ZIP)**

☑ Keep me informed about BI news and upcoming articles with a bi-weekly newsletter (uncheck if you prefer to proceed without signing up for the newsletter)

☑ Send me SQLBI promotions (only 1 or 2 emails per year)

By downloading the file(s), you agree to our Privacy Policy (https://www.sqlbi.com/privacy/) and reCAPTCHA Privacy Policy (https://policies.google.com/privacy).

WRITTEN BY

**Marco Russo & Alb**
**(https://www.sqlb**

Marco Russo and Alberto Ferrari are the founders of
regularly publish articles about Microsoft Power BI, D
SQL Server Analysis Services.

Marco and Alberto have worked with Analysis Servic
Pivot since the first versions, becoming established e
wrote 10 books on these technologies and provide c
mentoring. They are also regular speakers at major ir
conferences, including Microsoft Ignite, Data Insight S
and SQLBits. And of course, they are qualified trainer
classes taught so far.

PUBLISHED FEB 4, 2026
FIRST VERSION SEP 16, 2025

Search for more articles, blog posts, and videos SEARCH

MAR 25, 2026

**DAX user-defined functions (UDF) vs. calculation groups (https://www.sqlbi.com/articles/dax-user-defined-functions-udf-vs-calculation-groups/)**

This article describes the different roles of user-defined functions and calculation groups, explaining when to use either, and how they complement each other in the design of a semantic model.

READ MORE (HTTPS://WWW.SQLBI.COM/ARTICLES/DAX-USER-DEFINED-FUNCTIONS-UDF-VS-CALCULATION-GROUPS/)

MAR 24, 2026

**Mastering DAX video course, 3rd edition (https://www.sqlbi.com/articles/mastering-dax-video-course-3rd-edition/)**

The third edition of the Mastering DAX video course is available!

READ MORE (HTTPS://WWW.SQLBI.COM/ARTICLES/MASTERING-DAX-VIDEO-COURSE-3RD-EDITION/)

MAR 9, 2026

## Creating functions for the like-for-like DAX pattern (https://www.sqlbi.com/articles/creating-functions-for-the-like-for-like-dax-pattern/)

This article offers a comprehensive guide to changing the like-for-like pattern into model-independent functions to enhance flexibility and simplify DAX code.

READ MORE (HTTPS://WWW.SQLBI.COM/ARTICLES/CREATING-FUNCTIONS-FOR-THE-LIKE-FOR-LIKE-DAX-PATTERN/)

FEB 25, 2026

## Debugging DAX variables using TOJSON and TOCSV (https://www.sqlbi.com/articles/debugging-dax-variables-using-tojson-and-tocsv/)

This article describes how to use the TOJSON and TOCSV functions to inspect the content of intermediate table variables when debugging a DAX measure.

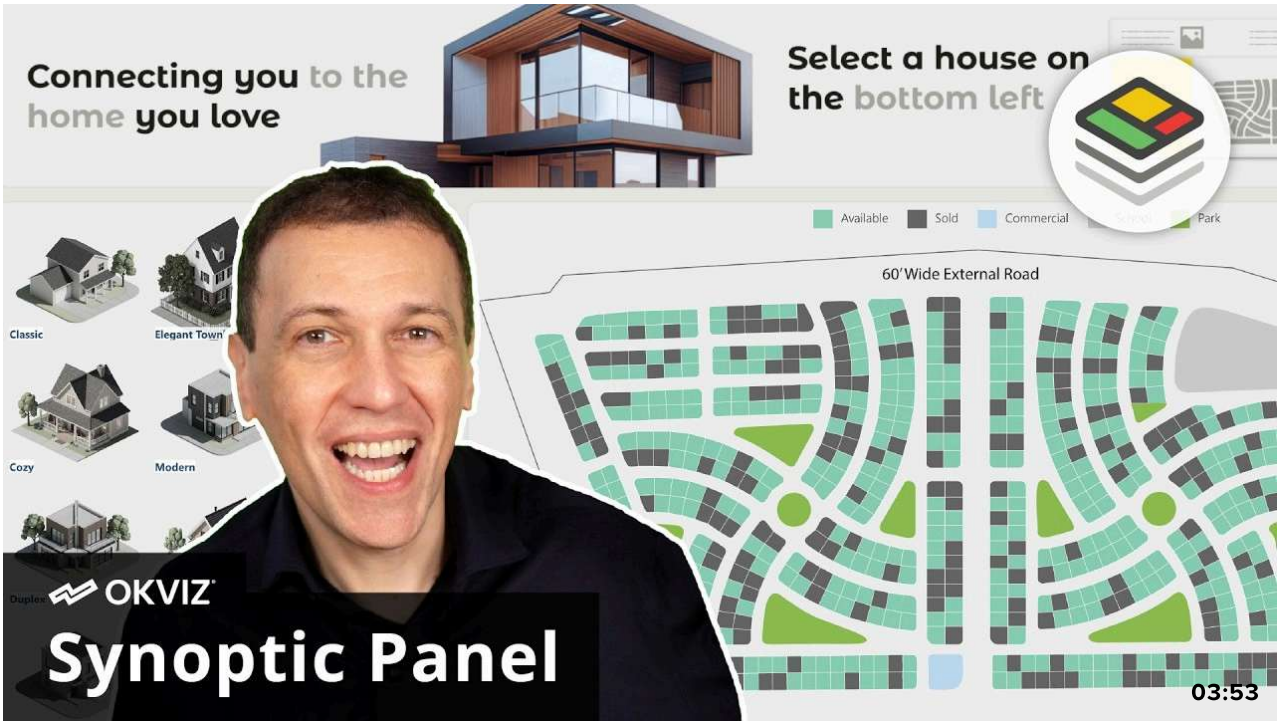
READ MORE (HTTPS://WWW.SQLBI.COM/ARTICLES/DEBUGGING-DAX-VARIABLES-USING-TOJSON-AND-TOCSV/)

(https://www.sqlbi.com/tv/creating-functions-for-the-like-for-like-dax-pattern/)

MAR 10, 2026

## Creating functions for the like for like DAX pattern (https://www.sqlbi.com/tv/creating-functions-for-the-like-for-like-dax-pattern/)

WATCH NOW (HTTPS://WWW.SQLBI.COM/TV/CREATING-FUNCTIONS-FOR-THE-LIKE-FOR-LIKE-DAX-PATTERN/)




(https://www.sqlbi.com/tv/real-estate-with-synoptic-panel-by-okviz/)

MAR 8, 2026

## Real estate with Synoptic Panel by OKVIZ (https://www.sqlbi.com/tv/real-estate-with-synoptic-panel-by-okviz/)

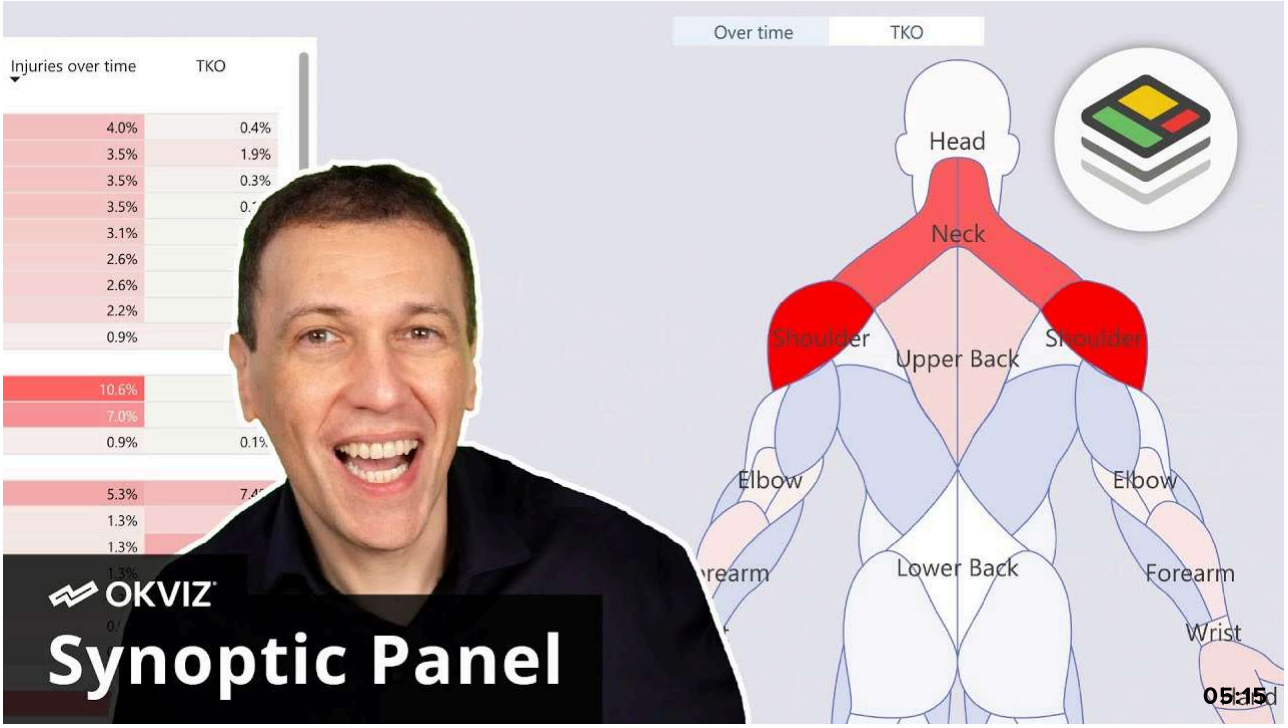
WATCH NOW (HTTPS://WWW.SQLBI.COM/TV/REAL-ESTATE-WITH-SYNOPTIC-PANEL-BY-OKVIZ/)

(https://www.sqlbi.com/tv/debug-dax-variables-using-tojson-and-tocsv/)

FEB 24, 2026

## Debug DAX variables using TOJSON and TOCSV (https://www.sqlbi.com/tv/debug-dax-variables-using-tojson-and-tocsv/)

WATCH NOW (HTTPS://WWW.SQLBI.COM/TV/DEBUG-DAX-VARIABLES-USING-TOJSON-AND-TOCSV/)




(https://www.sqlbi.com/tv/displaying-injury-data-on-the-human-body-by-using-synoptic-panel-in-power-bi/)

FEB 19, 2026

## Displaying injury data on the human body by using Synoptic Panel in Power BI (https://www.sqlbi.com/tv/displaying-injury-data-on-the-human-body-by-using-synoptic-panel-in-power-bi/)

WATCH NOW (HTTPS://WWW.SQLBI.COM/TV/DISPLAYING-INJURY-DATA-ON-THE-HUMAN-BODY-BY-USING-SYNOPTIC-PANEL-IN-POWER-BI/)

## Comments

66 Comments

Login ▼

G

Join the discussion…

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name

Share

Best Newest Oldest

A

**Ali Shaan**

6 months ago

Not Easy but Good People to Fly with :)

1 0 Reply

C

**Cody T**

2 months ago

I have been using this along with chapter 13 of your new book and found it extremely helpful.

## Newsletter

Your email address

SUBSCRIBE

By subscribing, you agree to our Privacy Policy (https://www.sqlbi.com/privacy/) and reCAPTCHA Privacy Policy (https://policies.google.com/privacy).

**275.8K**
subscribers

Get BI news and original content in your inbox every 2 weeks!

NEWSLETTER ARCHIVE (HTTPS://WWW.SQLBI.COM/NEWSLETTER/)

## What is ne

- Mastering DAX (https://www.sqlbi.c dax-workshop-hous 1-2026/) – Mar 30, H
- Mastering DAX (https://www.sqlbi.c dax-workshop-new- 10-2026/) – Apr 8, N
- Mastering DAX (https://www.sqlbi.c dax-workshop-cope 7-2026/) – May 5, C
- Mastering DAX (https://www.sqlbi.c dax-workshop-balti 29-2026/) – May 27
- Mastering DAX (https://www.sqlbi.c dax-workshop-zuric 2026/) – Jun 10, Zü

- Mastering DAX (https://www.sqlbi.c dax-workshop-berli 2026/) – Jun 15, Be
- Mastering DAX (https://www.sqlbi.c dax-workshop-amst jul-2-2026/) – Jun 3

---

(https://www.sqlbi.com/) (https://dax.guide/)

(https://www.daxpatterns.com/) (https://daxlib.org)

(https://dax.do/) (https://www.daxformatter.com/)

(https://www.daxoptimizer.com/) (https://bravo.bi)